

FORGE GROWTH

Installation Guide

Every way to install it — a server from published images, a server from source, a Mac or Windows laptop — then attaching a domain and connecting WhatsApp. Every command can be copied as-is.

Repository github.com/Forgemind-git/ForgeGrowth-OSS

Revision `main`

Contents

0	Before you start — what the machine needs
1	Server, from published images — the usual route; nothing is compiled
2	Attaching a domain — both kinds of server
3	Connecting WhatsApp — Meta credentials and the webhook
4	Server, from source — git clone, build locally
5	Local install — macOS
6	Local install — Windows
7	ngrok — WhatsApp against a laptop
8	Upgrading, backup, removal
9	When it goes wrong — real errors and what they mean

Which section do I need?

Situation	Read
Installing on a customer's server	1 → 2 → 3
Server already runs Traefik / nginx / another site	1 → 2b → 3
You intend to change the code	4
Trying it on your own laptop	5 or 6, then 7
Already installed, moving to a newer version	8

One rule that saves the most time

The directory you are standing in is what selects the install. Every command in this guide is run from the install directory, because one machine can hold several installs and they are told apart by nothing else.

0. Before you start

Everything runs in containers. You do not install Node, PostgreSQL, Redis or MinIO on the machine.

Requirement	Detail
Docker	with the Compose v2 plugin. <code>docker compose version</code> must work — note the space, not <code>docker-compose</code>
Shell	Linux, macOS, or Windows through WSL2 / Git Bash. There is no PowerShell installer
RAM	2 GB only if building from source (section 4). The published-image route has no build step
Disk	~3 GB. First download is about 350 MB compressed
Ports	one for the web UI (8080 by default; the installer moves to the next free one). Ports 80 and 443 as well, if this machine is to obtain its own HTTPS certificate

Check the machine is ready

```
docker compose version
docker run --rm hello-world
```

If the second command prints a greeting, Docker works. If either fails, fix that before continuing — every failure after this point will be blamed on the installer instead.

What gets created

The installer makes one directory — `forge-growth` — containing six small files and no source code:

```
forge-growth/
.env           your settings and every generated secret
.forgegrowth-install which version this install is pinned to
docker-compose.yml  downloaded; overwritten on every upgrade
caddy/Caddyfile    only used when this machine handles HTTPS itself
up.sh  down.sh      start and stop
install.sh          a copy of the installer, for upgrades
```

Your data is not in that folder. It lives in Docker volumes, which is why `./down.sh` is safe and `docker compose down -v` destroys everything:

```
docker volume ls | grep forgegrowth
```

1. Server, from published images

The normal way to install on a customer's machine. Nothing is cloned, nothing is compiled, and every secret is generated for you.

Step 1 — run the installer

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)"
```

It asks a single question — the address people will use to reach it — and derives everything else.

Step 2 — or answer up front, so it asks nothing

The machine owns ports 80 and 443, and DNS already points at it

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)" -- \
  --domain crm.example.com
```

A Let's Encrypt certificate is obtained on the way up, and the installer confirms the certificate actually works before reporting success.

Something else already owns 80 and 443 (Traefik, nginx, another site)

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)" -- \
  --url https://crm.example.com --yes
```

Use `--url`, not `--domain`. It tells the installer that HTTPS is terminated elsewhere, so it will not start a second web server to fight over the ports. Then follow section 2b.

No domain yet — reach it by IP

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)" -- \
  --url http://203.0.113.10:8080 --yes
```

A plain-HTTP address is fine to start with, but it cannot connect WhatsApp

Meta refuses any webhook address that is not `https://`. Add a domain later (section 2) — no reinstall is needed.

Step 3 — pin the version on a customer machine

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/Forgemind-git/ForgeGrowth-OSS/v1.4.0/scripts/install.sh)" -- \
  --version v1.4.0 --domain crm.example.com
```

Both halves matter: the URL fixes which script runs, and `--version` fixes what that script then downloads and pulls. The choice is remembered, so a later bare re-run cannot silently drift onto the newest build.

All the flags

Flag	Meaning
<code>--domain <host></code>	Serve HTTPS here, certificate included. Needs ports 80 + 443 free
<code>--url <origin></code>	Public address when something else terminates HTTPS
<code>--port <n></code>	Host port for the UI (default 8080, or the next free one)
<code>--dir <path></code>	Install here instead of <code>./forge-growth</code>
<code>--version <ref></code>	Pin the downloaded files and the image tag together
<code>--admin-email <addr></code>	First admin account (default <code>admin@<your domain></code>)
<code>--source</code>	Build from a checkout rather than pulling images
<code>--yes</code> , <code>-y</code>	Never ask. With no address flag that means localhost

Step 4 — sign in

The installer prints the URL, the admin email and the password. If the password shows as unchanged, read it from the file:

```
cd forge-growth
grep '^BOOTSTRAP_ADMIN_PASSWORD=' .env
```

Managing the stack afterwards

Run these **from the install directory** — that is what picks the install.

Command	Does
<code>./up.sh</code>	Start, then verify the public URL really answers
<code>./down.sh</code>	Stop. Keeps all data
<code>./install.sh</code>	Upgrade in place
<code>docker compose logs -f backend</code>	Follow the backend log
<code>docker compose ls</code>	List every install on this machine
<code>docker compose down -v</code>	Deletes all data. Take a dump first

Use up.sh and down.sh, not docker compose up -d

The scripts perform the only two checks that catch the failures this software actually suffers: a site that returns 404 while every container reports healthy, and a second folder quietly adopting this install's database. Raw Compose skips both.

2. Attaching a domain

Which instructions apply depends on one thing: does this install own ports 80 and 443, or does something else? The installer already worked it out and recorded the answer.

```
cd forge-growth
grep '^TLS_MODE=' .env
```

Value	Meaning	Follow
TLS_MODE=caddy	This install owns the ports and can obtain its own certificate	2a
TLS_MODE=proxy	Something else owns them; that thing must route the domain here	2b

2a — This install owns ports 80 and 443

1. Point the domain's **A record** at the server's IP address.
2. In the app: **Admin Settings → Domain**, type the hostname, press **Add**.
3. Open `https://crm.example.com` in a browser.

The certificate is obtained during that first visit. Nothing restarts and no file is edited.

Setting it at install time instead:

```
./install.sh --domain crm.example.com
```

2b — Something else owns the ports

Adding the domain in the app is still worth doing first — it makes the API accept requests from that hostname immediately. What it cannot do is obtain a certificate, because it does not hold the ports. Your existing proxy must accept the hostname and forward it to this install's **web container on port 80**.

Traefik, running in Docker on the same host

Create an overlay file outside the install directory — `docker-compose.yml` is re-downloaded on every upgrade, so labels added there are eventually lost:

```
nano /opt/forge-growth-proxy.yml
```

```

services:
  web:
    networks: [default, proxy_default]
    labels:
      - traefik.enable=true
      - traefik.docker.network=proxy_default

      # ONE service, named explicitly on every router below.
      - traefik.http.services.forgegrowth.loadbalancer.server.port=80

      - "traefik.http.routers.forgegrowth.rule=Host(`crm.example.com`)"
      - traefik.http.routers.forgegrowth.entrypoints=websecure
      - traefik.http.routers.forgegrowth.tls=true
      - traefik.http.routers.forgegrowth.tls.certresolver=myresolver
      - traefik.http.routers.forgegrowth.service=forgegrowth

networks:
  proxy_default:
    external: true

```

Find the network Traefik is actually on, and replace `proxy_default` with it:

```

docker inspect <traefik-container> \
  --format '{{range $k,$v := .NetworkSettings.Networks}}{{ $k }}\n'}}{{end}}'

```

Wire it in through `.env`, then recreate only the web container:

```

cd forge-growth
echo 'COMPOSE_FILE=docker-compose.yml:/opt/forge-growth-proxy.yml' >> .env
docker compose up -d web

```

Traefik reads the labels from the Docker socket. It is not restarted and no Traefik file changes.

Two mistakes here cost an afternoon each

One service per router. Declaring a `loadbalancer.server.port` for each router makes Traefik unable to choose, so it discards every router on that container and each hostname returns 404 from its catch-all — including one that worked a minute earlier. Declare one service and name it with `.service=` on each router.

No certresolver behind a CDN. If the domain is proxied by Cloudflare or similar, the ACME challenge stops at the CDN and can never succeed. It then retries forever, and those failures are rate-limited per account — enough of them stop every other domain on that server from renewing. Use `tls=true` with no resolver and let the CDN supply the certificate.

nginx, running outside Docker

```
server {
    listen 443 ssl;
    server_name crm.example.com;

    ssl_certificate      /etc/letsencrypt/live/crm.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/crm.example.com/privkey.pem;

    client_max_body_size 100M;          # media uploads; nginx defaults to 1M

    location / {
        proxy_pass http://127.0.0.1:8080;    # WEB_PORT from .env

        proxy_set_header Host                $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host    $host;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;

        proxy_http_version 1.1;
        proxy_set_header Upgrade      $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 600s;
    }
}
```

2c — Prove it actually works

Admin Settings → Domain, then press **Check** on the row. The server fetches the address from the outside and reports which link in the chain is broken.

Result	Meaning
Working over HTTPS	Done — and it reached this install, not another one
Reachable over plain HTTP only	Routing is right; the certificate is missing
Something answered, but it is not this install	The proxy has no route for it — usually a typo in the host rule
A different application answered	The domain points at another app, or an older copy of this one
Another install answered	Several installs on this machine; the domain is on the wrong one
That domain does not resolve	DNS

A page that loads in your browser is not proof

It shows the interface was served. It does not show the API accepts requests from that address — and that is the half which breaks sign-in. Press Check.

3. Connecting WhatsApp

The app does nothing useful until a WhatsApp Business number is attached. This needs an `https://` address, so finish section 2 first.

Step 1 — collect four values from Meta

At **developers.facebook.com** → your app → **WhatsApp** → **API Setup**:

Value	Where
Phone number ID	API Setup, under the number
WhatsApp Business Account ID	Same screen
App ID	App dashboard header
Access token	API Setup. The one shown there expires in 24 hours — create a System User token in Business Settings for anything permanent

Step 2 — add the number in the app

Admin Settings → **WhatsApp Accounts** → **Add**. Paste the display number (digits only, no `+` and no spaces), the Phone Number ID, the WABA ID, the App ID and the token. The token is encrypted at rest with AES-256-GCM.

Step 3 — read your webhook verify token

```
cd forge-growth
grep '^META_WEBHOOK_VERIFY_TOKEN=' .env
```

Step 4 — test the handshake before touching Meta

This is the exact request Meta sends when you press Verify and save. Running it yourself turns a slow, silent failure into a one-second answer:

```
cd forge-growth
TOKEN=$(grep '^META_WEBHOOK_VERIFY_TOKEN=' .env | cut -d= -f2-)
curl -s "https://crm.example.com/api/webhook/whatsapp?
hub.mode=subscribe&hub.verify_token=$TOKEN&hub.challenge=42"
```

```
42
```

If it prints `42`, Meta will verify successfully. Anything else — fix it here first.

Step 5 — point Meta at it

WhatsApp → Configuration → Webhook → Edit:

Field	Value
Callback URL	<code>https://crm.example.com/api/webhook/whatsapp</code>
Verify token	the <code>META_WEBHOOK_VERIFY_TOKEN</code> from step 3

Save, then under **Webhook fields → Manage**, subscribe to **messages**. That single field covers messages, delivery statuses, echoes and template events.

The path is `/api/webhook/whatsapp`

Not `/api/webhook`. A mismatch gives a silent 404 at verification time and no useful message from Meta.

Step 6 — confirm

- **Admin Settings → Webhooks → Send Test Webhook** → "Incoming text message". Expect an audit row with `processed` and `records_extracted=1`.
- Send a real WhatsApp message to the number. It should appear in **Chats**.
- Send one from the app to confirm the outbound direction.

One callback address per WhatsApp Business Account

Every phone number under one WABA shares a single callback URL. Pointing it at a new install takes inbound messages away from whatever had it before — instantly and with no warning. If you are moving a live number, plan it as a cutover, not a configuration change.

4. Server, from source

Use this only if you intend to change the code. It builds the images on the machine, which needs about 2 GB of free memory and a few minutes.

```
git clone https://github.com/ForgeMind-git/ForgeGrowth-OSS.git
cd ForgeGrowth-OSS
./scripts/install.sh --domain crm.example.com
```

The same flags apply as in section 1. Without a domain:

```
./scripts/install.sh --yes
```

Managing a source install

Command	Does
<code>./scripts/up.sh</code>	Start and verify the public URL
<code>./scripts/down.sh</code>	Stop, keeping data
<code>git pull && ./scripts/install.sh</code>	Upgrade
<code>./scripts/uninstall.sh</code>	Remove, including all data

After changing code

```
docker compose build backend web
docker compose up -d
```

A source install does not upgrade by pulling images

`docker compose pull` has nothing to fetch — the images are built locally. Changes reach the running site only when you **rebuild**. A `git pull` alone moves the files on disk and nothing else, which looks exactly like a feature failing to arrive.

Running the tests

```
cd backend && npm test && npm run lint
cd frontend && npm run test:unit && npm run lint
```

Roughly a third of the backend tests need a database and quietly skip without one. To make that an error instead:

```
cd backend && REQUIRE_DB=1 npm test
```

5. Local install — macOS

Works on both Apple Silicon and Intel. The published images carry `linux/amd64` and `linux/arm64`, so nothing is emulated.

Step 1 — Docker

Install Docker Desktop, OrbStack or Colima, start it, then confirm:

```
docker compose version
```

Step 2 — install

```
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)" -- --yes
```

`--yes` means "localhost, ask me nothing". It creates `~/forge-growth` and serves on port 8080 — or the next free port, which it tells you.

Step 3 — sign in

```
cd ~/forge-growth
grep '^BOOTSTRAP_ADMIN_PASSWORD=' .env
```

Open the URL it printed and sign in as `admin@example.com`.

The first download is about 350 MB and shows progress slowly

Roughly one to five minutes depending on the connection. To watch it from another terminal: `docker system df`

Everyday commands

```
cd ~/forge-growth
./up.sh                # start
./down.sh              # stop, keeping data
docker compose logs -f backend  # follow the log
open ~/forge-growth    # reveal in Finder (Cmd+Shift+. shows .env)
```

6. Local install — Windows

The installer is a shell script, so it runs inside a Unix shell. WSL2 is the recommended route; Git Bash also works.

Step 1 — install WSL2

In PowerShell **as Administrator**:

```
wsl --install
```

Reboot when prompted, then open **Ubuntu** from the Start menu and create your Linux user.

Step 2 — Docker Desktop

Install Docker Desktop, then enable **Settings → Resources → WSL integration** for your distro. Confirm from inside the Ubuntu terminal:

```
docker compose version
```

Step 3 — install, from the Ubuntu terminal

```
cd ~
bash -c "$(curl -fsSL https://raw.githubusercontent.com/ForgeMind-git/ForgeGrowth-OSS/main/scripts/install.sh)" -- --yes
```

Install into the Linux filesystem, not the Windows drive

Use `~/forge-growth`. A location under `/mnt/c/...` is dramatically slower and confuses file-watching. `cd ~` first, every time.

Step 4 — sign in

```
cd ~/forge-growth
grep '^BOOTSTRAP_ADMIN_PASSWORD=' .env
```

Open the printed URL in your normal Windows browser — WSL2 forwards `localhost`.

Reaching the files from Windows

Type this in Explorer's address bar, or run from the Ubuntu terminal:

```
explorer.exe .
```

```
\\wsl$\\Ubuntu\\home\\<your-user>\\forge-growth
```


7. ngrok — WhatsApp against a laptop

Meta will not accept a `localhost` or plain-HTTP callback, so a laptop needs a public HTTPS address borrowed from a tunnel.

Use a Meta test number, never a live one

Every Meta app includes a free test number on its own WhatsApp Business Account, so its callback is independent of any production number. A laptop also sleeps, and inbound messages arriving then are lost rather than queued.

Step 1 — install and authenticate

```
brew install ngrok # macOS
ngrok config add-authtoken <your-token> # dashboard.ngrok.com -> Your Authtoken
```

On Windows, install ngrok inside WSL2 so it can see the same ports.

Step 2 — claim a static domain

dashboard.ngrok.com → Domains → New Domain. The free tier includes one. Without it the URL changes on every restart and the Meta webhook has to be re-pointed each time.

Step 3 — start the tunnel

Use the port your install actually reported:

```
ngrok http --url=your-name.ngrok-free.app 8080
```

Leave that terminal running and open a new one for the rest.

Step 4 — confirm the tunnel reaches the app

```
curl -s -o /dev/null -w '%{http_code}\n' https://your-name.ngrok-free.app/
```

```
200
```

Step 5 — run Meta's handshake yourself

```
cd ~/forge-growth
TOKEN=$(grep '^META_WEBHOOK_VERIFY_TOKEN=' .env | cut -d= -f2-)
curl -s "https://your-name.ngrok-free.app/api/webhook/whatsapp?
hub.mode=subscribe&hub.verify_token=$TOKEN&hub.challenge=42"
```

This works before any WhatsApp account is configured

The check falls back to the token in `.env`, so you can prove the tunnel and the handshake on their own — before credentials are in play. That separates "my tunnel is wrong" from "my credentials are wrong", which is most of the difficulty.

Step 6 — allow the tunnel address in the app

Admin Settings → Domain → add `your-name.ngrok-free.app` → press **Check**. Required if you open the interface through the tunnel rather than through localhost.

Step 7 — configure Meta and test

Field	Value
Callback URL	<code>https://your-name.ngrok-free.app/api/webhook/whatsapp</code>
Verify token	the value from step 5
Subscribe to	<code>messages</code>

Add your own phone under **API Setup → To → Manage phone number list**; a test number will only message people on that list. Then watch every delivery arrive:

```
http://127.0.0.1:4040
```

That is ngrok's inspector. It shows each request Meta makes with the full body, and lets you replay them — the most useful tool in this loop by a distance.

Things that will bite

- Closing the ngrok terminal kills the tunnel silently.
- The token on Meta's API Setup screen expires after **24 hours**. When inbound stops for no apparent reason, this is usually why.
- Free-tier ngrok shows a warning page to browsers. Meta's requests are unaffected; you will only see it if you open the interface through the tunnel.

8. Upgrading, backup, removal

Upgrading

Re-running the installer is the upgrade. From the install directory:

```
cd forge-growth
./install.sh
```

It re-downloads the compose file and scripts, pulls the newest images, applies any new database migrations at start-up, and restarts. `.env` is never overwritten, and secrets are kept.

To move deliberately to a specific version:

```
./install.sh --version v1.5.0
```

For a source install:

```
git pull && ./scripts/install.sh
```

A change to the installer itself takes effect on the next run

The run that downloads a new `install.sh` is still executing the old copy from memory. Normally invisible — but it is why an upgrade occasionally needs running twice.

Backup — the one thing you cannot recreate

```
cd forge-growth
grep -E '^(FORGECRM_ENCRYPTION_KEY|POSTGRES_PASSWORD)= ' .env
```

Store these two off the server

`FORGECRM_ENCRYPTION_KEY` is the only thing that decrypts stored Meta, Google and payment-gateway credentials. `POSTGRES_PASSWORD` is applied only when the database directory is first created, so a regenerated `.env` cannot open an existing database. Lose either and the failure names the wrong thing — an authentication error that reads like a network problem.

Database dump

```
cd forge-growth
docker compose exec -T postgres pg_dump -U forgegrowth forgegrowth \
| gzip > backup-$(date +%F).sql.gz
```

Removal

```
cd forge-growth
docker compose down -v          # stops and DELETES all data
cd .. && rm -rf forge-growth
```

9. When it goes wrong

"no matching manifest for linux/arm64/v8"

An image with no build for your processor. Compose pulls concurrently and reports one failure against every service, so the message names `postgres`, `redis`, `minio` and `caddy` — all of which are innocent. Check the two images this project publishes:

```
docker manifest inspect ghcr.io/forgemind-git/forgrowth-web:latest | grep architecture
```

To carry on regardless, run everything under emulation:

```
export DOCKER_DEFAULT_PLATFORM=linux/amd64
```

"denied" or 403 while downloading images

A GHCR package is created **private** even when its repository is public. The owner has to publish both packages, once:

```
Packages -> forgrowth-backend -> Package settings -> Change visibility -> Public
Packages -> forgrowth-web      -> Package settings -> Change visibility -> Public
```

It logs me out on every refresh, or says Unauthorized when I change tab

The login cookie is marked `Secure` and the site is being served over `http://`, so the browser silently discards it. Sign-in returns success and the next request is rejected, with nothing in any log.

Reach the site over `https://`. If HTTPS is terminated by a proxy this stack cannot see, set the flag explicitly:

```
cd forge-growth
echo 'COOKIE_SECURE=true' >> .env
./up.sh
```

Check what the app itself believes under **Admin Settings → Domain**.

Every container is healthy but the site returns 404

Nothing is routing the domain to the install. From inside the stack nothing is wrong, which is why container health cannot detect it. Press **Check** in Admin Settings → Domain, and see section 2b.

"port 8080 is in use"

Not an error — the installer moved to the next free port and told you which. Find what holds a port:

```
lsof -nP -iTCP:8080 -sTCP:LISTEN      # macOS
ss -lntp | grep 8080                  # Linux
```

A container keeps restarting

Usually a memory ceiling set too low — a container that reaches its limit is killed and restarted, which looks like a container that never finishes starting.

```
cd forge-growth
docker compose ps
docker compose logs --tail 50 backend
```

Raise the ceiling in `.env` — `BACKEND_MEM_LIMIT` first, since its appetite grows with conversation volume and media downloads. A busy install wants `1g`.

A second install adopted the first one's database

Installs on one machine are separated by their project name and nothing else. Check which directory owns which:

```
docker compose ls
grep '^COMPOSE_PROJECT_NAME=' forge-growth/.env
```

`up.sh` refuses to start when another directory already owns this install's name, which is the reason to use it rather than raw Compose.

Useful one-liners

```
cd forge-growth

grep '^BOOTSTRAP_ADMIN_PASSWORD=' .env      # admin password
grep '^META_WEBHOOK_VERIFY_TOKEN=' .env     # webhook verify token
grep '^TLS_MODE=' .env                      # caddy = owns 80/443, proxy = does not
grep '^WEB_PORT=' .env                      # which port the UI is on

docker compose ps                           # container states
docker compose logs -f backend              # follow the backend
docker compose ls                           # every install on this machine
docker volume ls | grep forgegrowth         # where the data lives
```